

תהליכים בלינוקס

קריאות מערכת לעבודה עם תהליכים
ניהול תהליכים בגרעין לינוקס
תורי תהליכים

תוכנית, תהליך וניהול התהליך

current

```
struct task_struct{
    int pid;
    unsigned long state;
    int prio;
    unsigned long policy;
    struct task_struct* parent;
    struct list_head tasks;
    struct list_head sibling;
    struct thread_struct thread;
    struct files_struct* files;
    .
    .
}
```

FDT -file descriptor table

stdin
stdout
stderr
HW1.txt
params.ini

struct thread_struct

הכתובת שאליה המעבד צריך לקפוץ לאחר שתהליך זה ייבחר לריצה
כתובת של האיבר האחרון במחסנית
•
•

./test.bin

0x2000

```
#include <stdio.h>
int main() {
    unsigned char ch;
    printf("Hello World\n");
    ch = getchar();
    while(1)
    {
        printf("char is %c",ch);
        sleep(1);
    }
    return 0;
}
```

./test.bin

0x5000

```
#include <stdio.h>
int main() {
    unsigned char ch;
    printf("Hello World\n");
    ch = getchar();
    while(1)
    {
        printf("char is %c",ch);
        sleep(1);
    }
    return 0;
}
```

קריאות מערכת לעבודה עם תהליכים

איך תהליך מגיע לעולם ?





קריאת המערכת `fork()`

`pid_t fork () ;`

- פעולה: מעתיקה את תהליך האב לתהליך הבן וחוזרת בשני התהליכים.
- קוד זהה (ומיקום בקוד).
- זיכרון זהה (משתנים וערכיהם, גם במחסנית וגם בערימה).
- סביבה זהה (קבצים פתוחים, ספריית עבודה נוכחית).
- אבל, תהליך הבן הוא תהליך נפרד מתהליך האב, לכן יש לו **PID משלו**.
- פרמטרים: אין.
- ערך מוחזר:
 - במקרה של כישלון: 1- לאב (אין בן).
 - במקרה של הצלחה: לבן מוחזר 0 ולאב מוחזר ה-**pid של הבן**.

איך נוכל להבדיל בין אב לבן אם ה-pid של הבן יוצא במקרה 0?

לפני fork()

parent

```
int main() {  
    int x = 0;  
    → pid_t p = fork();  
    if (p == 0) {  
        x = 1;  
    } else {  
        x = 2;  
    }  
}
```

אחרי fork()

parent

```
int main() {  
    int x = 0;  
    pid_t p = fork();  
    → if (p == 0) {  
        x = 1;  
    } else {  
        x = 2;  
    }  
}
```

son

```
int main() {  
    int x = 0;  
    pid_t p = fork();  
    → if (p == 0) {  
        x = 1;  
    } else {  
        x = 2;  
    }  
}
```

קוד האב וקוד הבן מסתעפים

parent

```
int main() {  
    int x = 0;  
    pid_t p = fork();  
    if (p == 0) {  
        x = 1;  
    } else {  
        → x = 2;  
    }  
}
```

son

```
int main() {  
    int x = 0;  
    pid_t p = fork();  
    if (p == 0) {  
        → x = 1;  
    } else {  
        x = 2;  
    }  
}
```

מה יהיה ערכו של x
בסופו של דבר?
1 או 2?



שכפול מרחב הזיכרון ע"י fork()

- לאחר פעולת fork() מוצלחת, אמנם יש לאב ולבן את אותם משתנים בזיכרון, אך **בעותקים נפרדים**.
- כלומר, שינוי ערכי המשתנים אצל האב לא ייראה אצל הבן, וההיפך.

הדפסה לא מתואמת למסך

- שאלה: מה מדפיס הקוד הבא אם fork נכשלת? ואם היא מצליחה?

- אם fork() נכשלה:

hello

- ואם fork() הצליחה?

- יש הרבה תשובות אפשריות, למשל:

hellohello

hheelloollo

helhelloolo

```
int main() {
    fork();
    printf("hello");
    return 0;
}
```

- הסיבה: שני התהליכים ניגשים בצורה לא מתואמת **למשאב משותף** - המסך.

קריאת המערכת wait()

```
pid_t wait(int *wstatus);
```

• פעולה: ממתנה עד אשר אחד מתהליכי הבן יסיים.

• פרמטרים:

- wstatus – מצביע למשתנה בו יאוחסנו פרטים על תהליך הבן שהסתיים.
- למשל, wstatus יכיל את ערך הסיום של הבן (הערך שהעביר כארגומנט ל-exit()).
ערך הסיום מופיע בבית השני מתוך ארבעת בתי ה-wstatus. כדי לחלץ אותו יש
לנצל את המקור WEXITSTATUS(*wstatus), המחזיר (*wstatus >> 8) & 0xff.
- במידה ולא מעוניינים בסטטוס הבן שסיים, אפשר להעביר NULL.

מבינים את החישוב?

• ערך מוחזר:

- אם אין בנים או שכל הבנים כבר סיימו ובוצע להם wait() – יוחזר מיד הערך 1-.
- אם יש בנים שסיימו ועדיין לא בוצע עבורם wait() (כלומר הם במצב **zombie** –
יפורט בשקופיות הבאות) – יוחזר מיד ה-pid של אחד הבנים הנ"ל.

איך תהליך אב יכול לחכות
לסיום כל תהליכי הבן?

• אחרת – המתנה עד שבן כלשהו יסיים.

הדפסה מתואמת למסך

- שימוש ב-`wait()` יכול לפתור את הבעיה שראינו קודם כאשר מדפיסים למסך במקביל משני תהליכים:

```
int main() {  
    pid_t p = fork();  
    if (p > 0) {  
        // parent waits for child  
        wait(NULL);  
    }  
    printf("hello");  
    return 0;  
}
```

קריאת המערכת `exit()`

```
void exit(int status);
```

- פעולה: מסיימת את ביצוע התהליך הקורא ומשחררת את כל המשאבים שברשותו. התהליך עובר למצב **zombie** עד שתהליך האב יבקש לבדוק את סיומו ואז יפונה לחלוטין.

מה המטרה של מצב זה?

• פרמטרים:

- `status` – ערך סיום המוחזר לאב אם יבדוק את סיום התהליך.
- בפועל ניתן להעביר להורה רק 8 ביטים בתור ערך סיום, ולכן קריאת המערכת תעביר `(status & 0xff)`.

• ערך מוחזר: הקריאה אינה חוזרת.

- לפי ה-man, קריאת המערכת `exit` לא יכולה להיכשל.



קריאת המערכת `exit()`

- שאלה: למה בכלל לקרוא ל-`exit(status)` , אם אפשר פשוט לרשום `return status` בסוף פונקציית ה-`main`?
- תשובה: `main` היא לא באמת הפונקציה הראשית של התכנית...

- `main()` נקראת ע"י `__libc_start_main()` שאוספת את ערך החזרה של `main()` וקוראת ל-`exit()`.

```
int __libc_start_main(...) {
    .....
    exit(main(...)) ;
}
```

- מסקנה: הפונקציה `exit` תמיד נקראת לסיום סטנדרטי של התוכנית.

גרסת ספריית libc:

- פונקציית ה- main שאנו מממשים באפליקציה אינה ה- Entry point של התוצר הבינארי!
- ישנו קובץ תלוי ארכיטקטורה בספריית ה- libc בשם start.S אשר מכיל את ה- Entry point לקובץ ההרצה.
- לרוב אנחנו מתלנקג'ים עם ספריית libc באופן דינאמי ולכן חייבת להיות התאמה בגרסה של קבצי ה- h של ה- libc אשר ב- toolchain לבין ספריית ה- libc אשר ב- Target.



```
typedef struct elf32_hdr{
    unsigned char e_ident[EI_NIDENT];    /* ELF "magic number" */
    Elf32_Half e_type;
    Elf32_Half e_machine;
    Elf32_Word e_version;
    Elf32_Addr e_entry;    /* Entry point virtual address */
    Elf32_Off e_phoff;    /* Program header table file offset */
    Elf32_Off e_shoff;    /* Section header table file offset */
    Elf32_Word e_flags;
    Elf32_Half e_ehsize;
    Elf32_Half e_phentsize;
    Elf32_Half e_phnum;
    Elf32_Half e_shentsize;
    Elf32_Half e_shnum;
    Elf32_Half e_shstrndx;
} Elf32_Ehdr;
```

מבנה
הקובץ
הבינארי
שמתקבל

קוד מתוך ספריית libc

```
_start:
.
.
call __libc_start_main
.
.
__libc_start_main(...){
.....
exit( main(...) );
}
```

תוכנית ב- C

```
int main()
{
    return 0;
}
```

X86 – 32bit

```
080483ed <main>:
80483ed: b8 00 00 00 00  mov $0x0,%eax
80483f2: c3              ret
```

בלמ"ס

קריאת המערכת `execv()`

```
int execv(const char *filename,  
           char *const argv[]);
```

• פעולה: טוענת תכנית חדשה לביצוע במקום התהליך הקורא.

• פרמטרים:

- `filename` – מסלול אל הקובץ המכיל את התכנית לטעינה.
- `argv` – מערך מצביעים למחרוזות המכיל את הפרמטרים עבור התכנית. האיבר הראשון מקיים `argv[0] == filename`, דהיינו מכיל את שם קובץ התכנית. האיבר שאחרי הפרמטר האחרון מכיל `NULL`.

• ערך מוחזר:

למה צריך `NULL`?

- במקרה של כישלון: -1.
- במקרה של הצלחה: הקריאה **אינה חוזרת**. איזורי הזיכרון (קוד, מחסנית, ...) של התהליך מאותחלים עבור התכנית החדשה שמתחילה להתבצע מההתחלה.

קריאת המערכת `execv()`

- מה ידפיס הקוד הבא?

```
int main() {  
    char *argv[] = {"date", NULL};  
    execv("/bin/date", argv);  
    printf("hello");  
    return 0;  
}
```

- התשובה:

- אם `execv()` מצליחה: את התאריך והשעה.
- אם `execv()` נכשלת: hello

דוגמת קוד מסכמת

פלט לדוגמה:

```
pid = 8919
child pid = 8920
Sun Oct 29
00:31:32 IDT 2017
parent pid = 8919
```

```
printf("pid = %d\n", getpid());

pid_t pid = fork();
if (pid == 0) {
    printf("child pid = %d\n", getpid());
    char* args[] = {"/bin/date", NULL};
    execv(args[0], args);
    printf("This should not be printed\n");
} else {
    wait(NULL);
    printf("parent pid = %d\n", getpid());
}
```

אתחול תהליכים בלינוקס

- משתמשים מתחברים לעבודה בלינוקס דרך מסופים (terminal).
- מסוף = מסך + מקלדת (מקומי או מרוחק).

- התהליך `init` יוצר תהליך בן עבור כל מסוף, אשר טוען ומבצע את המשימות הבאות לפי הסדר:

1. איתחול של המסוף.

2. התחברות של המשתמש עם שם משתמש וסיסמא באמצעות תכנית `login`.

3. אם אושרה כניסת המשתמש: קריאה לתוכנית **shell** (כמו `tcsh` או `bash`) המאפשרת למשתמש להעביר פקודות למערכת ההפעלה.

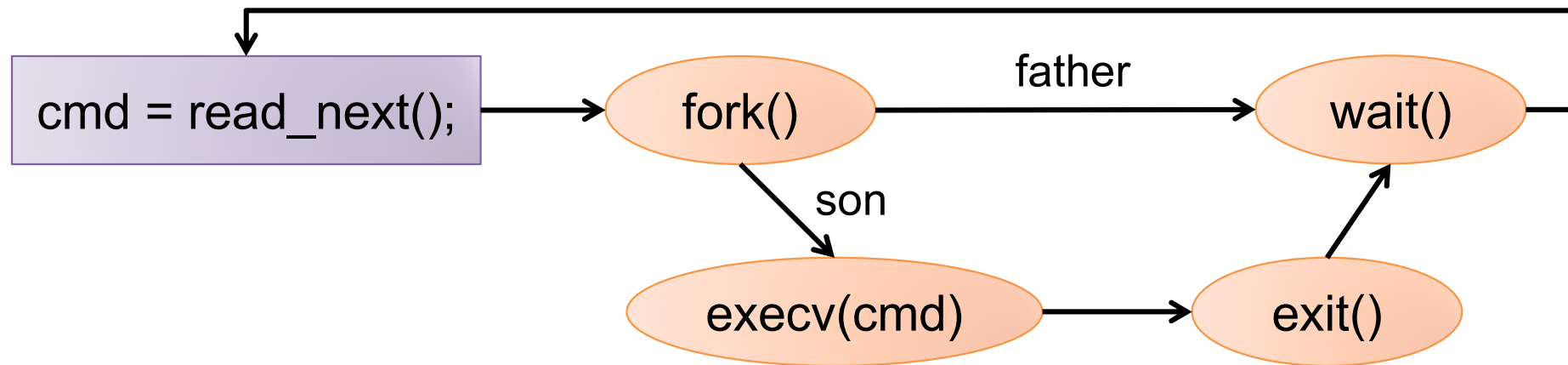
דוגמה לשימוש בתהליכים - shell

- ממשק שורת פקודה (command line).
 - ייעוד עיקרי: לקבל פקודות ולבצע אותן באופן סדרתי.
 - ה-shell מייצר תהליך בן עבור כל פקודה על-מנת לבצע אותה.
 - כל פקודה ניתן להריץ בחזית (foreground) או ברקע (background).
 - הרצה בחזית: האב (shell) ממתין לסיום הבן לפני קריאת הפקודה הבאה.
 - הרצה ברקע: האב (shell) עובר מיד לקריאת הפקודה הבאה.
 - ייעוד נוסף: להציג קבצים ותיקיות על-מנת לסייר במערכת.
- דוגמה חיה:
- https://www.tutorialspoint.com/unix_terminal_online.php

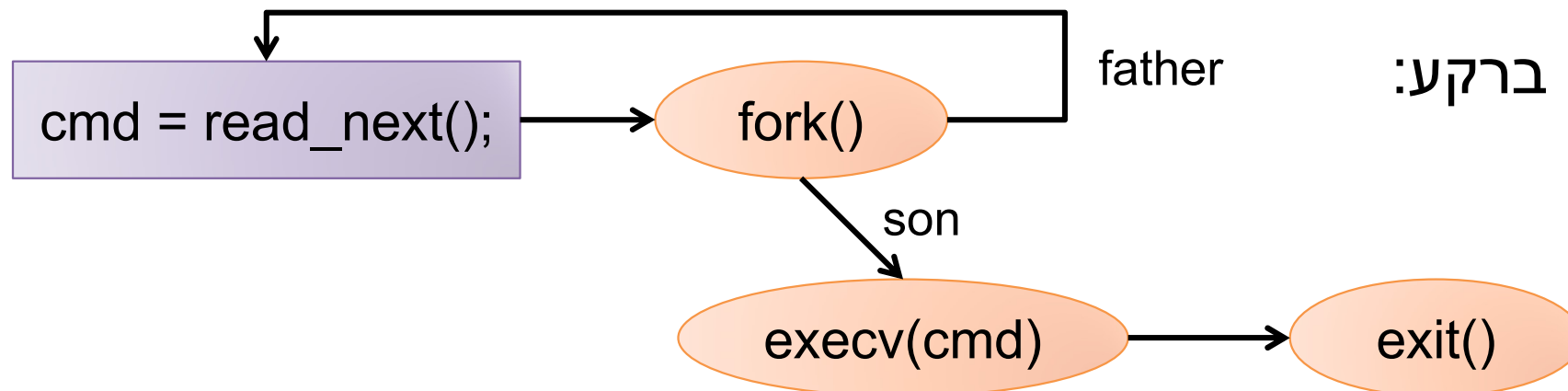


אופן פעולת shell בלינוקס

• הרצה בחזית:



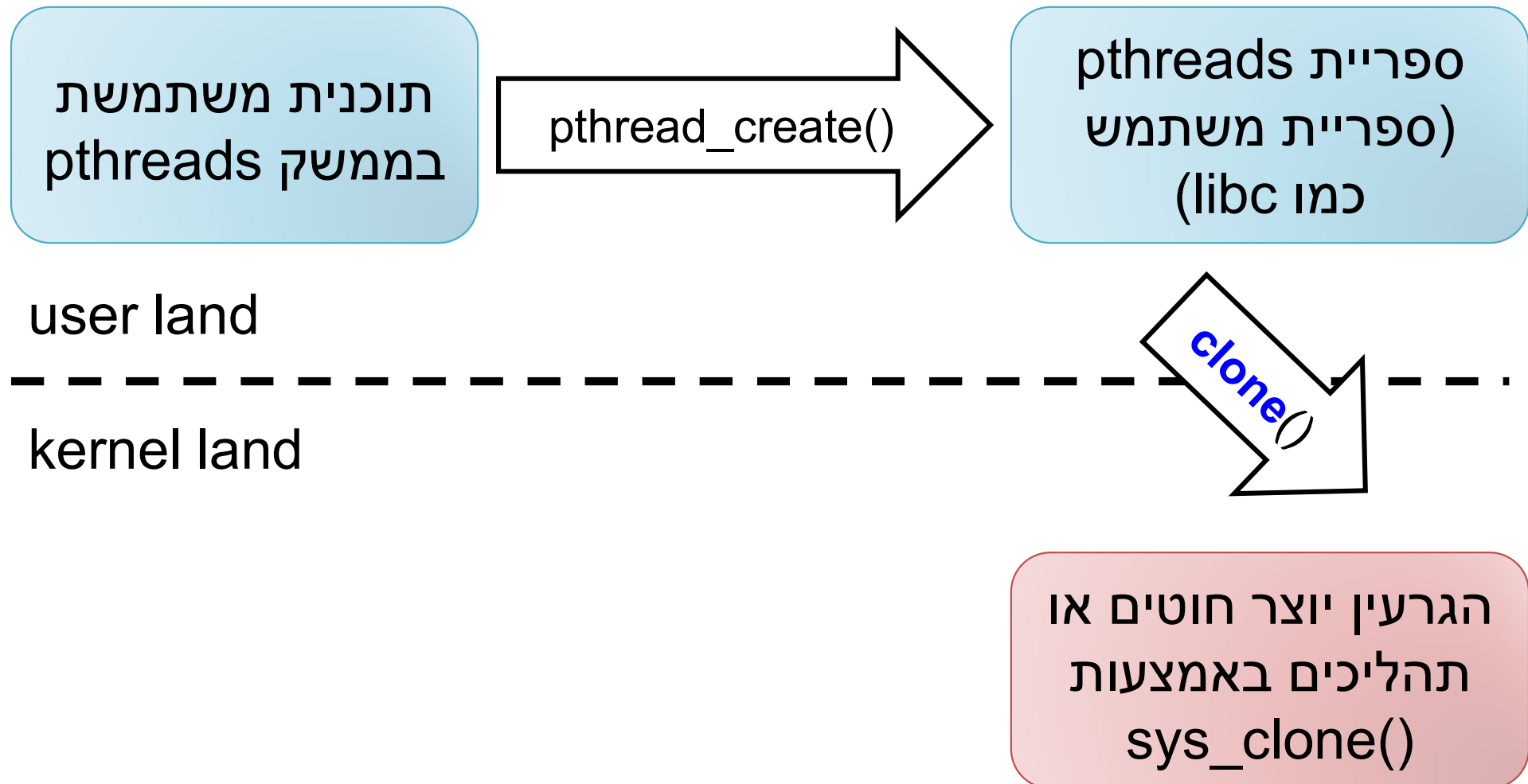
• הרצה ברקע:



תרגיל

יצירת טרמינל מנוון

תרשים: חוטים במבט מערכתי



קריאת המערכת clone()

```
int clone(int (*fn)(void *), void *child_stack,
          int flags, void *arg, ...
          /* pid_t *ptid, void *newtls, pid_t *ctid */ );
```

```
/* For the prototype of the raw system call, see NOTES */
```

DESCRIPTION

`clone()` creates a new process, in a manner similar to `fork(2)`.

This page describes both the glibc `clone()` wrapper function and the underlying system call on which it is based. The main text describes the wrapper function; the differences for the raw system call are described toward the end of this page.

Unlike `fork(2)`, `clone()` allows the child process to share parts of its execution context with the calling process, such as the virtual address space, the table of file descriptors, and the table of signal handlers. (Note that on this manual page, "calling process" normally corresponds to "parent process". But see the description of `CLONE_PARENT` below.)

One use of `clone()` is to implement threads: multiple flows of control in a program that run concurrently in a shared address space.

When the child process is created with `clone()`, it commences execution by calling the function pointed to by the argument `fn`. (This differs from `fork(2)`, where execution continues in the child from the point of the `fork(2)` call.) The `arg` argument is passed as the argument of the function `fn`.

When the `fn(arg)` function returns, the child process terminates. The integer returned by `fn` is the exit status for the child process. The child process may also terminate explicitly by calling `exit(2)` or after receiving a fatal signal.

קריאת המערכת clone()

```
int clone(int (*fn)(void*), void *child_stack,  
          int flags, void *arg);
```

- פעולה: יוצרת תהליך בן המשתף עם תהליך האב משאבים ונתונים לפי בחירה.

- פרמטרים:

- fn – מצביע לפונקציה שתהווה את הקוד הראשי של התהליך החדש.
- arg – הפרמטר המועבר לפונקציה fn() בתחילת ביצוע התהליך החדש.
- כשביצוע הפונקציה fn(arg) מסתיים, נגמר התהליך החדש.
- child_stack – מצביע לראש המחסנית של התהליך החדש.
- תזכורת: המחסנית גדלה לכיוון הכתובות הנמוכות.
- לכן child_stack צריך להצביע לסוף בלוק הזיכרון המוקצה לטובת המחסנית.

איזו מחסנית?
משתמש / גרעין?



קריאת המערכת clone()

- flags – מסכת דגלים הקובעת את צורת השיתוף בין התהליך הקורא והתהליך החדש. להלן מספר דגלים אופייניים:

שיתוף מרחב הזיכרון	CLONE_VM
שיתוף טבלת הקבצים הפתוחים	CLONE_FILES
שיתוף טבלת נתוני עבודה עם קבצים, המכילה נתונים כגון ספרית העבודה הנוכחית ועוד	CLONE_FS
לתהליך החדש יהיה אותו אב כמו התהליך הקורא (אחרת החדש יהיה הבן של הקורא)	CLONE_PARENT
התהליך החדש הוא חוט באותה קבוצת חוטים כמו התהליך הקורא (אותו tgid). גורר גם CLONE_PARENT	CLONE_THREAD

- ניתן לשלב מספר דגלים יחד באמצעות OR לוגי ביניהם, לדוגמה: `CLONE_FS | CLONE_VM`.

- ערך מוחזר: במקרה של הצלחה מוחזר ה-PID של התהליך החדש, אחרת -1.